Universidade Estadual de Mato Grosso do Sul
Ciência da Computação
Banco de Dados

Métodos de acesso

Prof. Nilton
nilton@comp.uems.br

Estruturas de índices para arquivos

- Estruturas auxiliares que acelera a velocidade de acesso aos registros
- Fornece caminhos de acesso secundários sem afetar o posicionamento físico dos registros
- Busca baseada em certos campos
  - Os campos de indexação
- Classificação
  - Índice ordenado de nível único
  - Índice multinível.

Índices ordenados de nível único

Índices ordenados de nível único

- Um campo é escolhido como campo de indexação
- Cada registro de índice
  - <valor campo de índice, endereço no arquivo>
- Busca binária no campo de índice
- O arquivo de índice é muito menor que o arquivo de dados.

Índices ordenados de nível único

- Tipos
  - Índices Primários
    - sobre campo chave de ordenação (não admite valores repetidos)
  - Índices Clusterizados (ou de agrupamento)
    - sobre um campo não chave de ordenação (admite valores repetidos)
  - Índices Secundários
    - sobre algum campo não ordenado do arquivo

- Caracterização
  - Índices densos
    - uma entrada de índice para cada registro
  - Índices não densos (ou esparsos)
    - uma entrada de índice para alguns registros

- Um arquivo ordenado pode ter quantos índices primários e clusterizados?

Índices primários

- Sobre campo chave de ordenação
- Índice não denso (ou esparso)
- O primeiro registro em cada bloco
  - registro âncora
- Número de entradas no arquivo de índice é igual ao número de blocos de disco do arquivo de dados ordenado
- Arquivo de índice ocupa menos espaço que o arquivo de dados
  - menos entradas
  - tamanho de cada registro da entrada é menor.

Índices primários

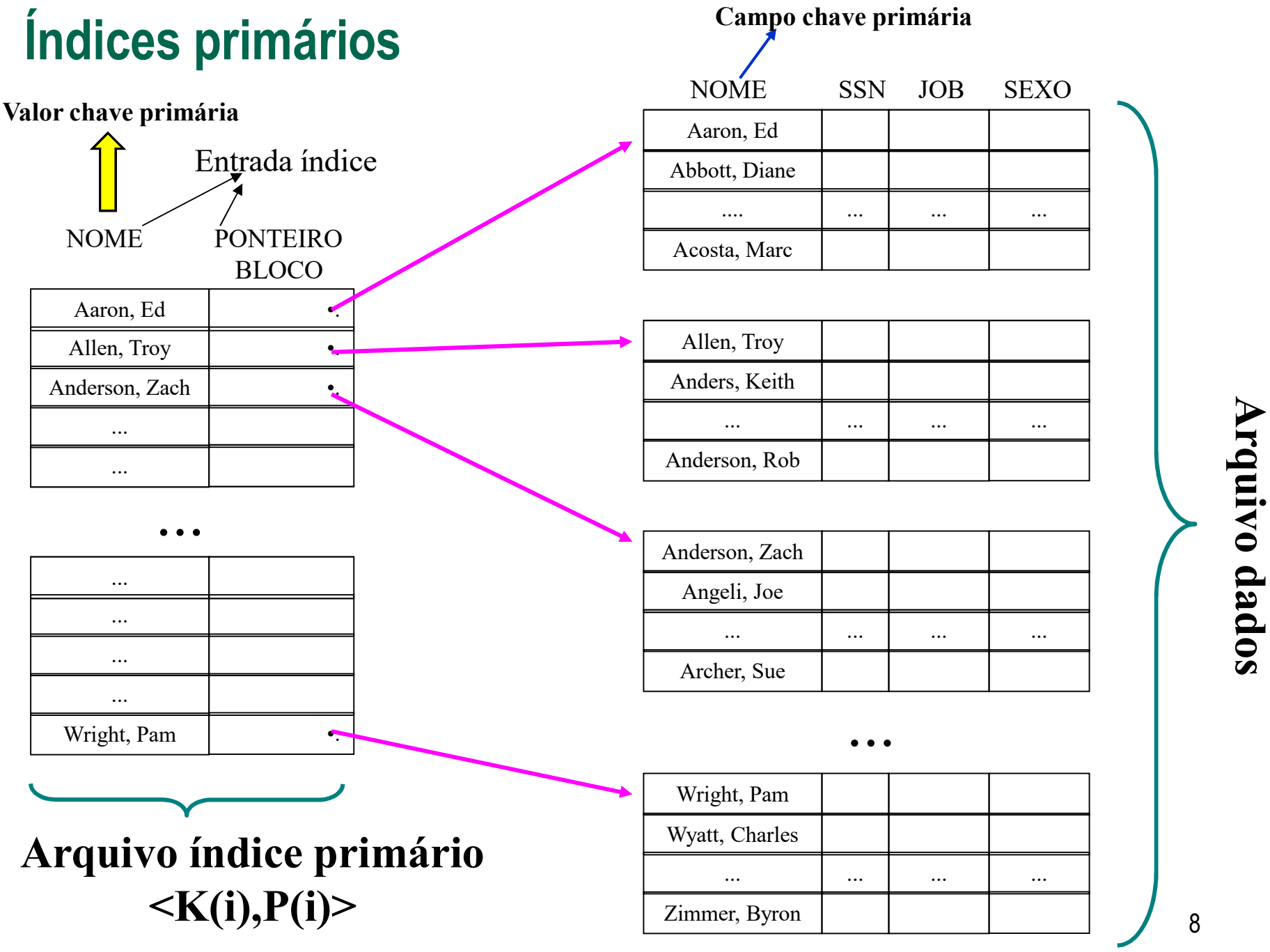
- Uma pesquisa binária em um arquivo de índice requer menos acessos a blocos do que uma pesquisa binária no arquivo de dados

- Se um arquivo de dados tiver *bi* blocos, então o custo de uma pesquisa binária no arquivo de dados é $\log_2 bi$ acessos de bloco

- Usando um arquivo de índice a recuperação de uma informação no arquivo de dados é $\log_2 bi + 1$, usando uma pesquisa binária

- Maior problema: inclusão e exclusão de registros
  - Na inclusão
    - Movimentar registros para abrir espaço no arquivo de dados
    - O registro âncora pode mudar
    - Uso de arquivo de overflow pode reduzir o problema
  - Na exclusão
    - Uso de marcadores de exclusão

Índices primários



Índices primários – exemplo 1

- **Suponha que temos um arquivo ordenado com**

r = 30.000 registros

B = 1.024 bytes (tamanho bloco)

R = 100 bytes (tamanho fixo do registro)

fb = chão(B / R) = chão(1.024 / 100) = 10 registros/bloco

b = teto(r / fb) = teto(30.000 / 10) = 3.000 blocos

A busca binária no arquivo de dados necessita aproximadamente $\lceil \log_2 b \rceil$ = $\lceil \log_2 3000 \rceil$ = 12 blocos acesso

* log 3000 / log 2 (transformar na base 10)

Índices primários – exemplo 1

- Agora suponha que temos um índice primário para o arquivo

V = 9 bytes longo (campo chave ordenação)

P = 6 bytes logo (ponteiro do bloco)

R_i = 9 + 6 = 15 bytes (tamanho de cada entrada)

fbr_i= chão(B / R_i) = chão(1024 / 15)= 68 entradas por bloco

r_i = 3.000 (nº entradas índice é igual ao número de blocos do arquivo de dados)

b_i=teto(r_i / fbr_i)= teto(3.000 / 68) = 45 blocos

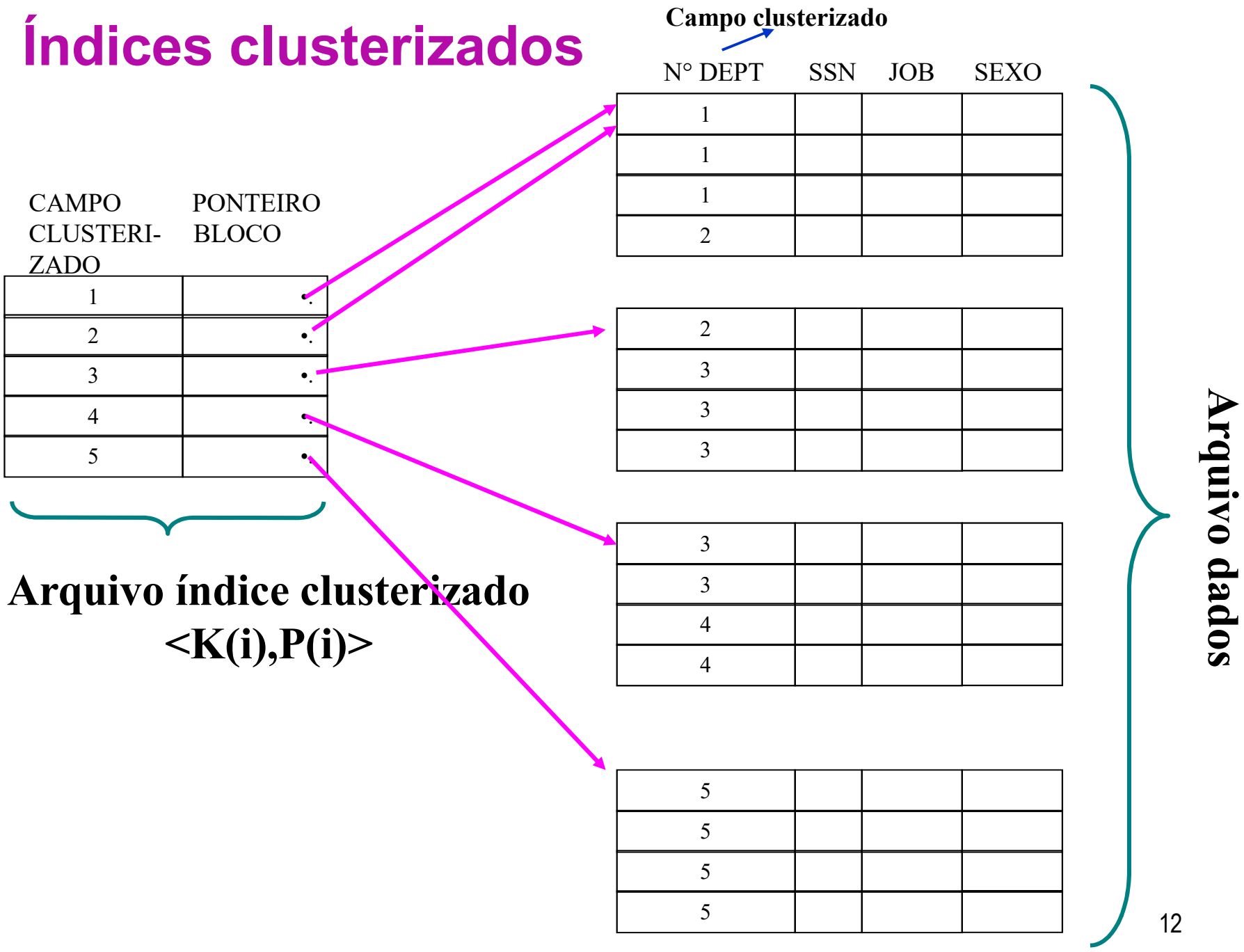
A busca binária usando índice necessita aproximadamente $\lceil \log_2 b_i \rceil$ = $\lceil \log_2 45 \rceil$ = 6 blocos acesso

Assim, temos 6 + 1 = 7 blocos acesso X 12 blocos acesso sem índice

Índices clusterizados

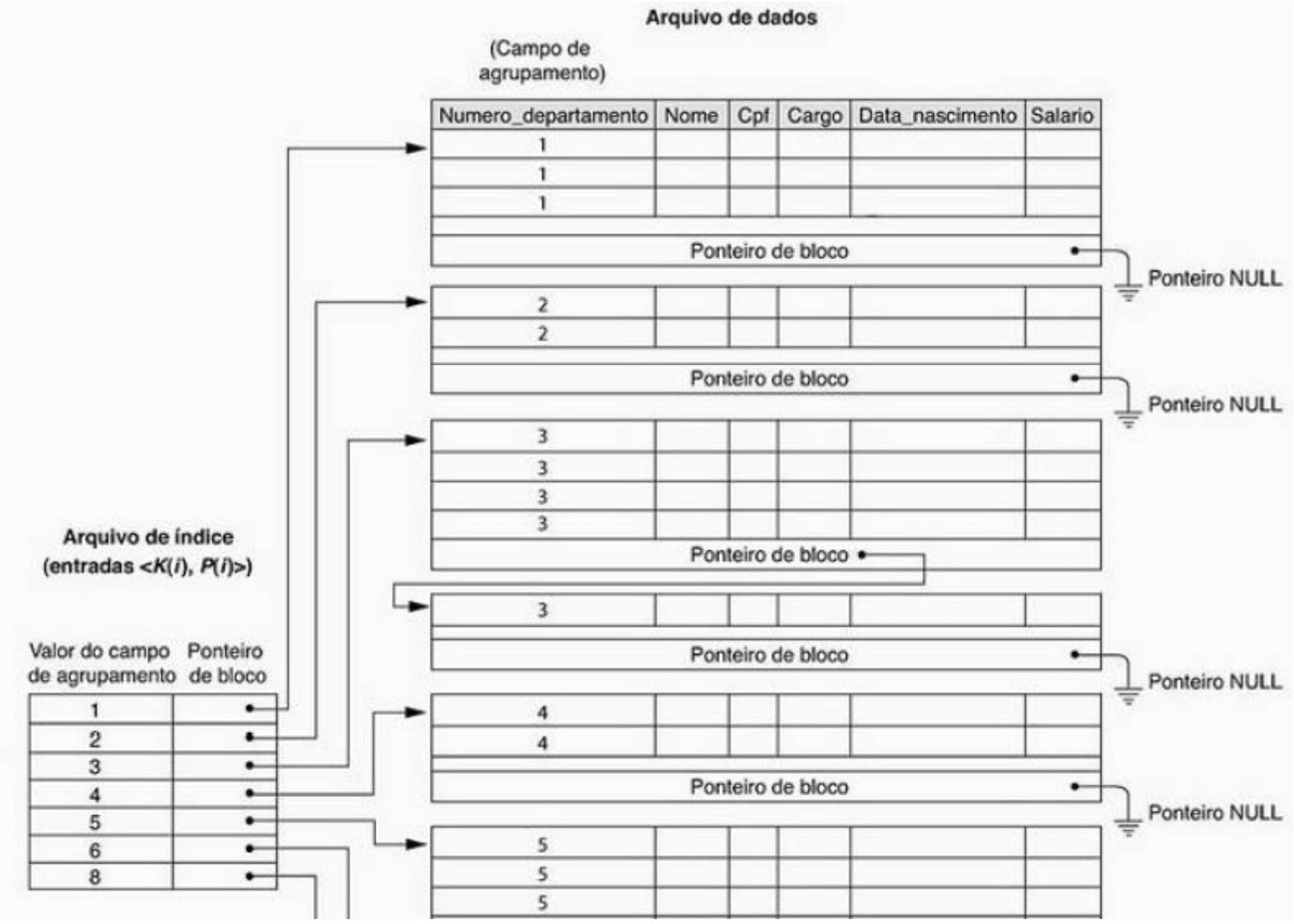
- Sobre campo não chave de ordenação
- Índice não denso (ou esparso)
- Uma entrada no índice para cada valor distinto do campo clustering
- A inclusão e exclusão ainda tem problemas
  - Minimiza se utilizar um bloco por valor distinto.

Índices clusterizados



Índices clusterizados

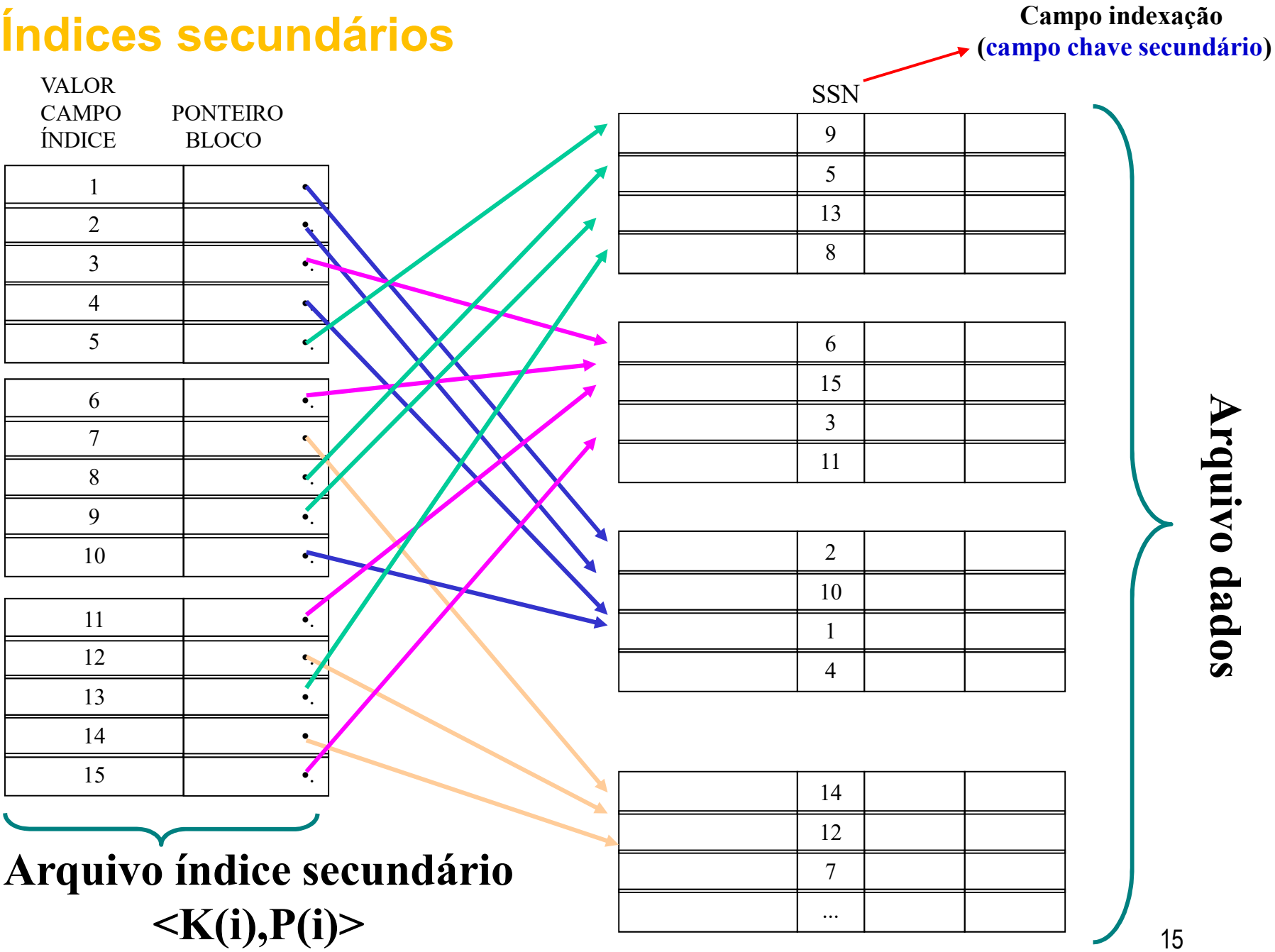
Um bloco por valor distinto: minimizar problemas na inclusão e exclusão



Índices secundários

- Sobre campo não ordenado do arquivo
  - Chave ou não chave
- Índice denso
- Podem existir vários índices secundários
- Não poder usar âncoras de bloco
  - O arquivo de dados não está fisicamente ordenado pelos valores do campo chave secundário
- Espaço de armazenamento
  - Maior que um índice primário
- Tempo de busca
  - Maior que um índice primário

Índices secundários



Índices secundários – exemplo 2

- Suponha que temos um arquivo ordenado

r = 30.000 registros (armazenados no disco)

B = 1.024 byte (tamanho bloco)

R = 100 bytes (tamanho fixo do registro)

fb = chão(B / R) = chão(1024 / 100) = 10 registros/bloco

b =teto(r / fb) = teto(30.000 / 10) = 3.000 blocos

Para pesquisa linear temos: b / 2 = 3.000 / 2 = 1.500 blocos acesso em média;

Índices secundários – exemplo 2

- **Agora suponha que temos um índice secundário para o arquivo num campo chave não ordenado**

  - V = 9 bytes longo (campo chave não ordenado)
  - P = 6 bytes logo (ponteiro do bloco)
  - R_i = 9 + 6 = 15 bytes (tamanho de cada entrada)
  - fbr_i = chão(B / R_i) = chão(1024 / 15) = 68 registros/bloco

  r_i = 30.000 (nº entradas índice denso)

  b_i = teto(r_i / fbr_i) = teto(30.000 / 68) = 442 blocos

  A busca binária usando índice necessita aproximadamente $\lceil \log_2 b_i \rceil$ = $\lceil \log_2 442 \rceil$ = 9 blocos acesso

Assim, temos 9 + 1 = 10 blocos acesso X 1.500 blocos de acesso na busca linear e menos rápido que o índice primário (10 X 7)

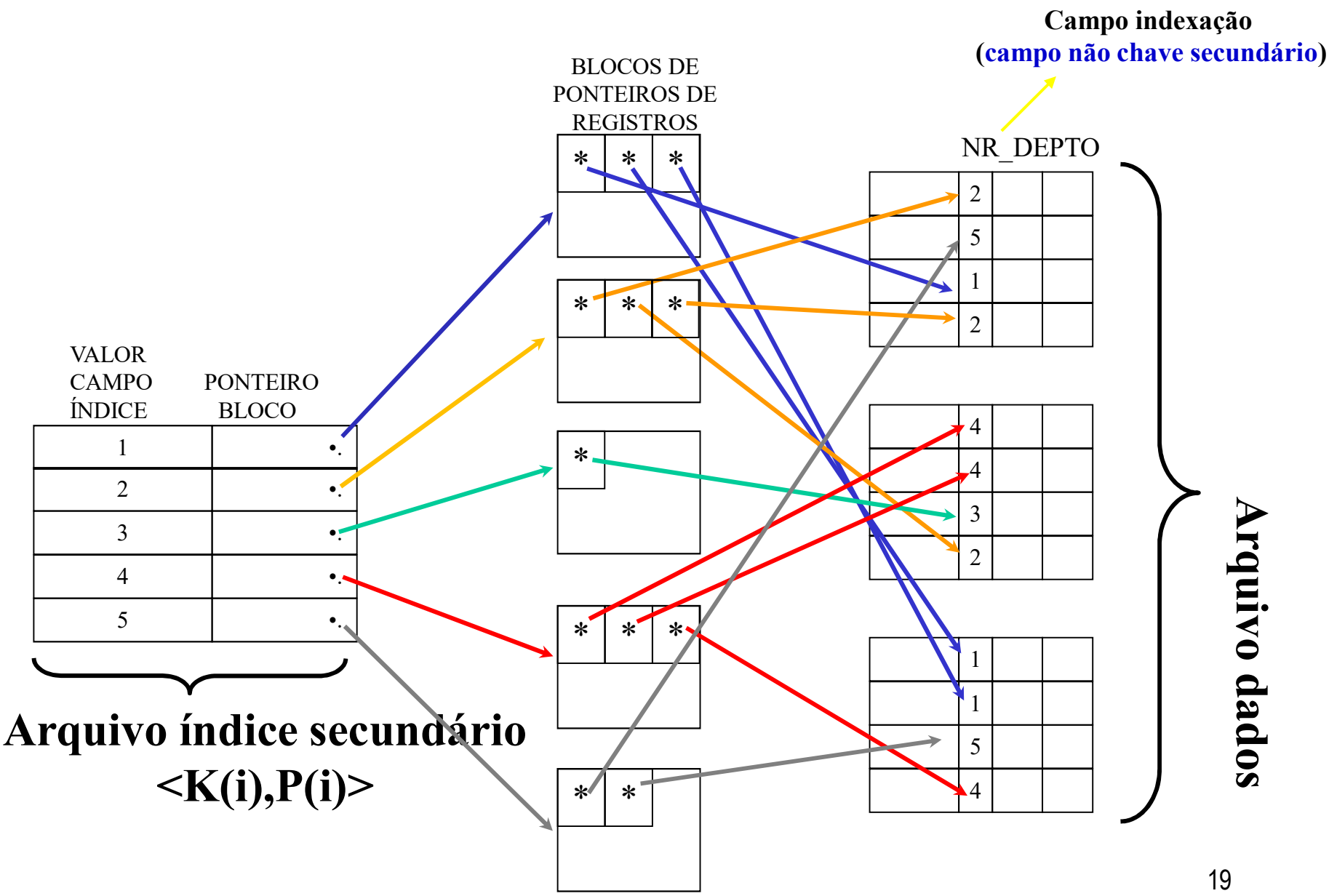
Índices secundários sobre campo não chave (aqui)

- **Opção 1:**
  - Uma entrada de índice para cada registro
  - Denso

- **Opção 2:**
  - Entradas de índice de tamanho variável: lista de ponteiros para as entradas repetidas (<K(i), P(i, 1), ..., P(i, k)>)

- **Opção 3 (mais comumente usada):**
  - Entradas de índice de tamanho fixo e uma só entrada para cada valor de campo índice com a criação de um nível de indireção extra para gerenciar múltiplos ponteiros
  - Não denso

Índices secundários sobre campo não chave (opção 3)



Índices ordenados nível único

Sumário

Tipos de Índices

	Campo Ordenação	Campo não Ordenado
Campo chave	Índice Primário	Índice Secundário (chave)
Campo não chave	Índice Clusterizado	Índice Secundário (não chave)

Índices ordenados nível único

Sumário

Propriedades dos Tipos de Índices

Tipo de Índice	Número de (Primeiro Nível) Entradas Índice	Denso ou Não Denso	Âncora Bloco no Arquivo de Dados
Primário	Número de blocos no arquivo de dados	Não Denso	Sim
Clusterizado	Número de valores do campo índice distintos	Não Denso	Sim/Não[a]
Secundário (chave)	Número de registros do arquivo de dados	Denso	Não
Secundário (não chave)	Número de registros (opção 1) ou número de valores campo índice distintos (opção 2 e 3)	Denso ou Não Denso	Não

[a]Sim se cada valor distinto do campo ordenação inicia um novo bloco, não caso contrário.

Índices de múltiplos níveis

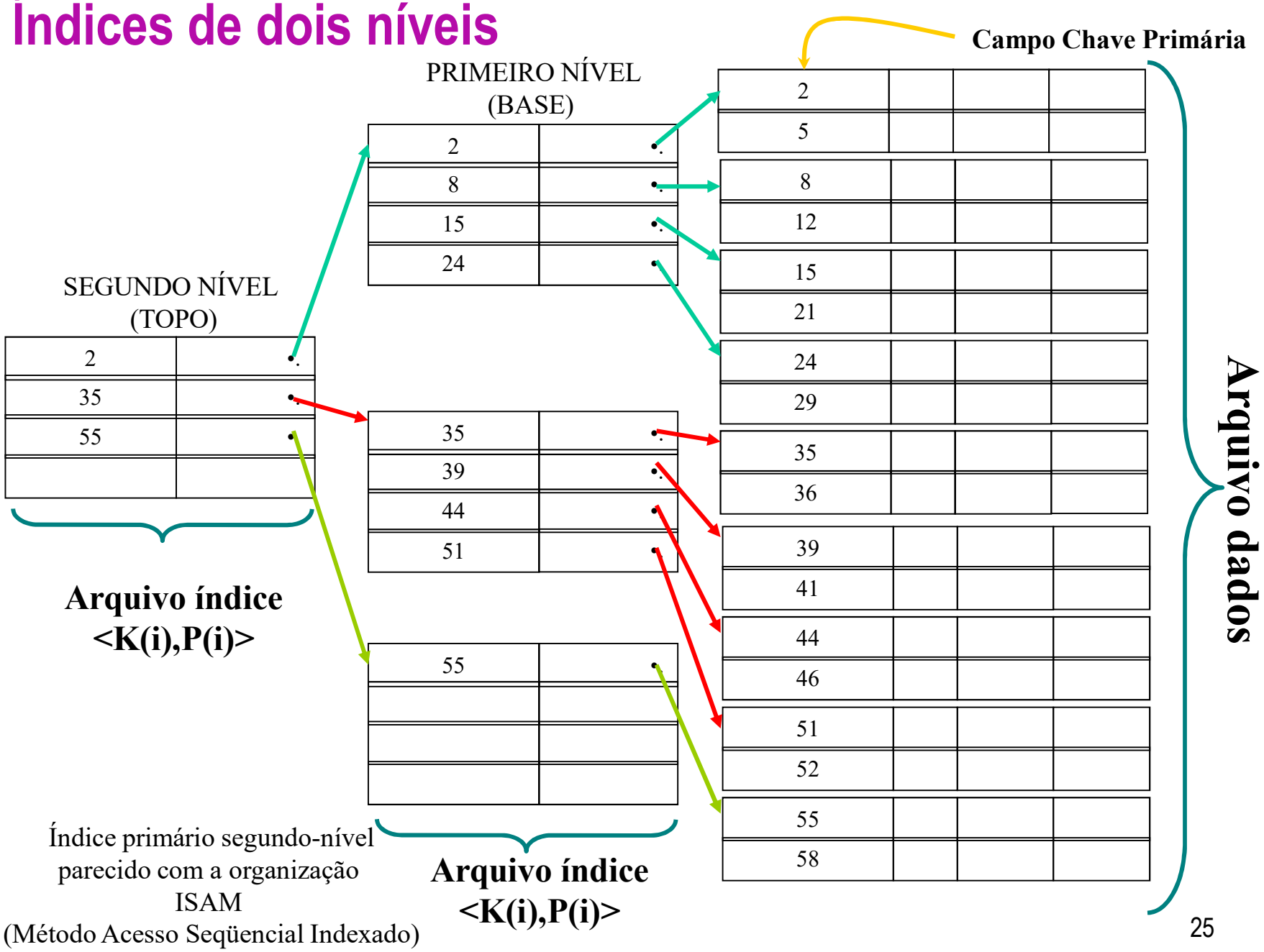
Índices de múltiplos níveis

- O esquema de indexação envolve arquivo de índice ordenado
- A busca binária pode ser utilizada neste arquivo e leva aproximadamente $\log_2 b_i$ blocos acesso, pois em cada passo do algoritmo é reduzido cada parte do arquivo índice pelo fator 2. Por isso a base é 2
- A idéia no índice multinível é reduzir o espaço de busca mais rapidamente, usando o valor do fbr_i do índice, o qual é maior que 2
- O fbr_i é chamado de fan-out (fo)
- Assim a busca vai exigir $\log_{fo} b_i$ blocos acesso, um número muito pequeno para a busca binária se o fan-out for um número maior que dois.

Índices de múltiplos níveis

- Criar um índice primário para o primeiro nível.
- Este índice para o primeiro nível é o segundo nível para o índice multiníveis.
- Como o segundo nível é um índice primário pode-se usar âncoras de bloco
- Um novo nível é acrescido caso exija mais de um bloco de armazenamento para as entradas do nível atual

Índices de dois níveis



Índices de múltiplos níveis – exemplo 3

- Suponha que temos um índice secundário denso (do exemplo 2)

fb_i = 68 entradas índice por bloco (é o fo)

b_1 = 442 blocos, do primeiro nível

b_2 = teto(b_1 / fo) = teto(442 / 68) = 7 blocos, do segundo nível

b_3 = teto(b_2 / fo) = teto(7 / 68) = 1 bloco, do terceiro nível, o nível topo

t = 3 (3 níveis)

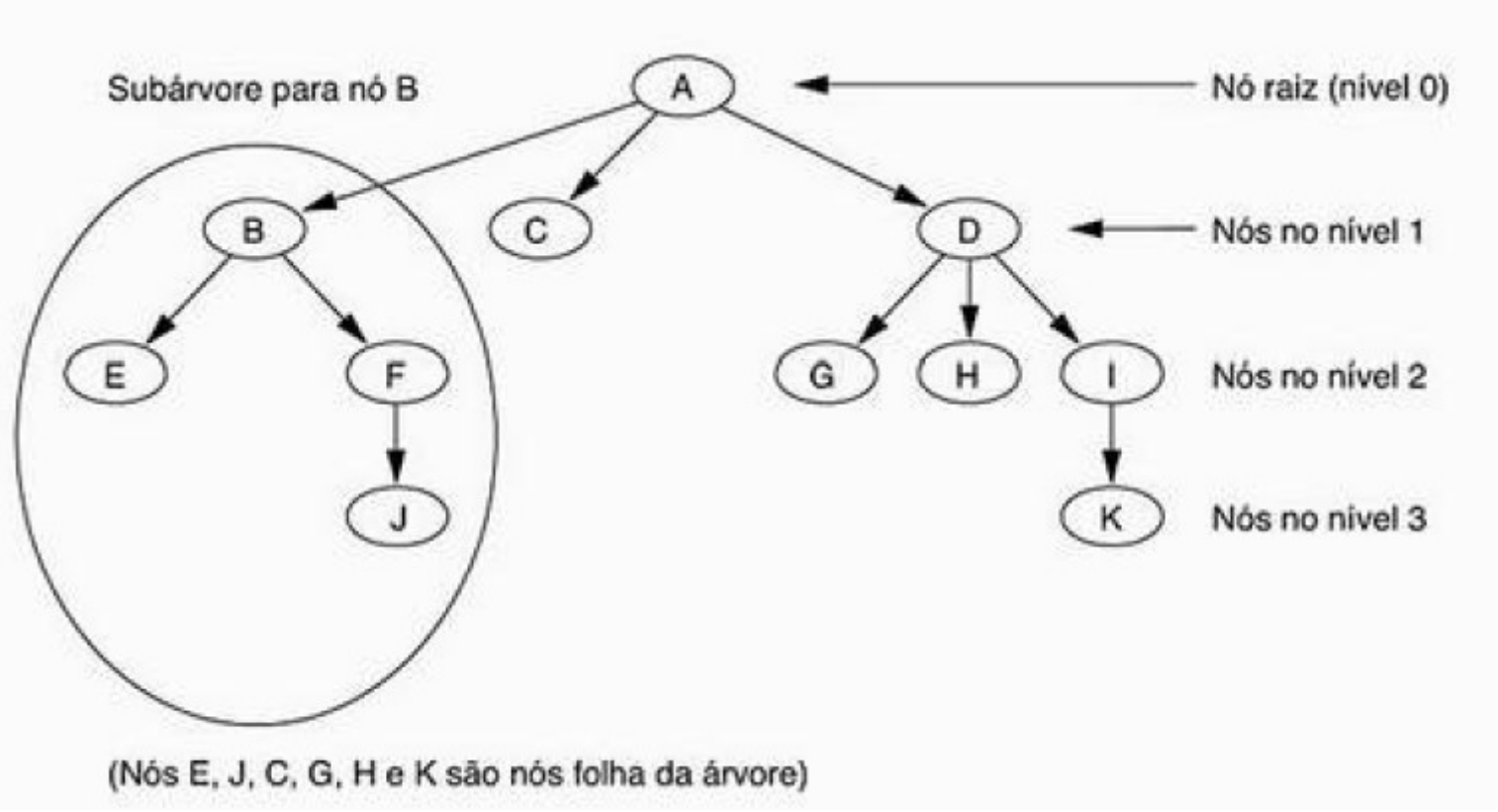
- **Para acesso a um registro usando o índice multiníveis: acessar um bloco a cada nível (3) mais um bloco no arquivo de dados (1), logo 4 acessos a bloco**

- **No exemplo: 4 acessos a bloco X 10 acessos a bloco usando índice nível único e pesquisa binária**

Índices de múltiplos níveis dinâmicos

- Busca eficiente
- Inserções e remoções
  - Continuam problemáticas, pois os índices são ordenados fisicamente
- Para contorná-los
  - o índice multiníveis dinâmico
    - Árvores de pesquisa
    - Árvores B
    - Árvores B+

Árvore



- Cada nós, exceto raiz, tem um nó pai e zero ou mais filhos
- Nó folha, nó interno, nível de um nó, descendente, subárvore, balanceamento

Árvore de pesquisa

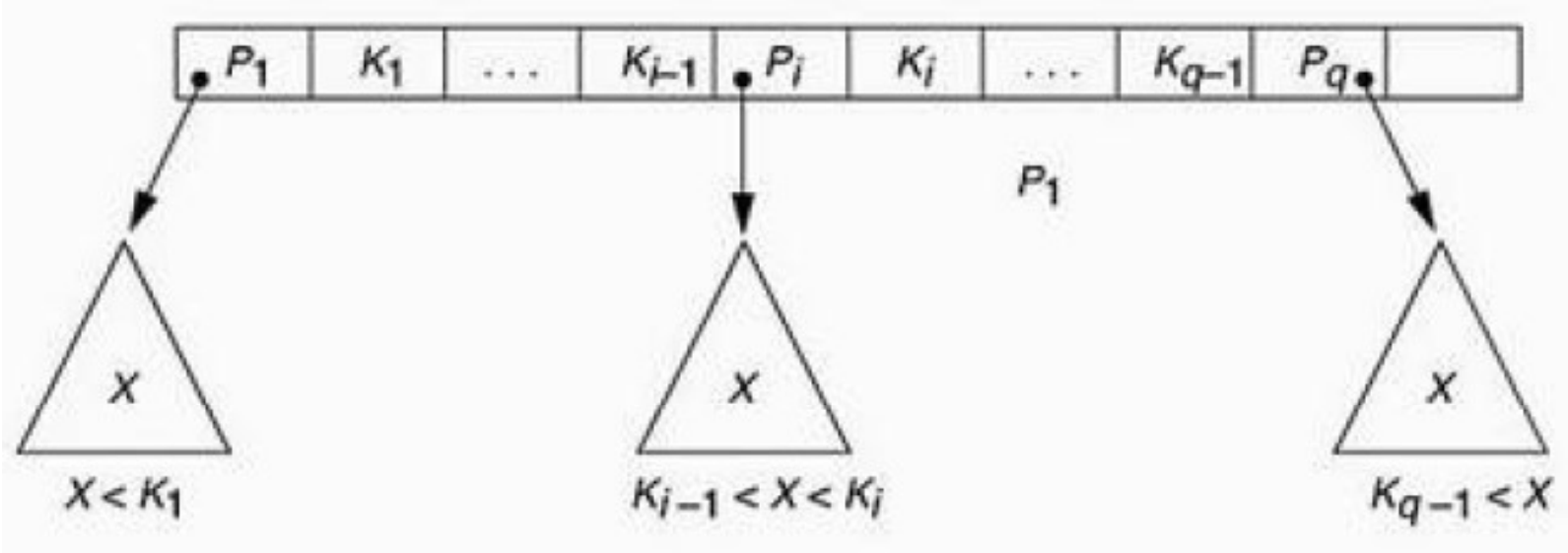
- Uma árvore de pesquisa de **ordem** P é uma árvore tal que cada nó contém no máximo P–1 valores de pesquisa e P ponteiros

$$< P_1, K_1, P_2, K_2, ..., P_{q-1}, K_{q-1}, P_q >, \text{ onde } q <= p$$

- Cada P_i é um ponteiro para um nó filho (ou NULL), e cada K_i é um valor de pesquisa de algum conjunto ordenado de valores

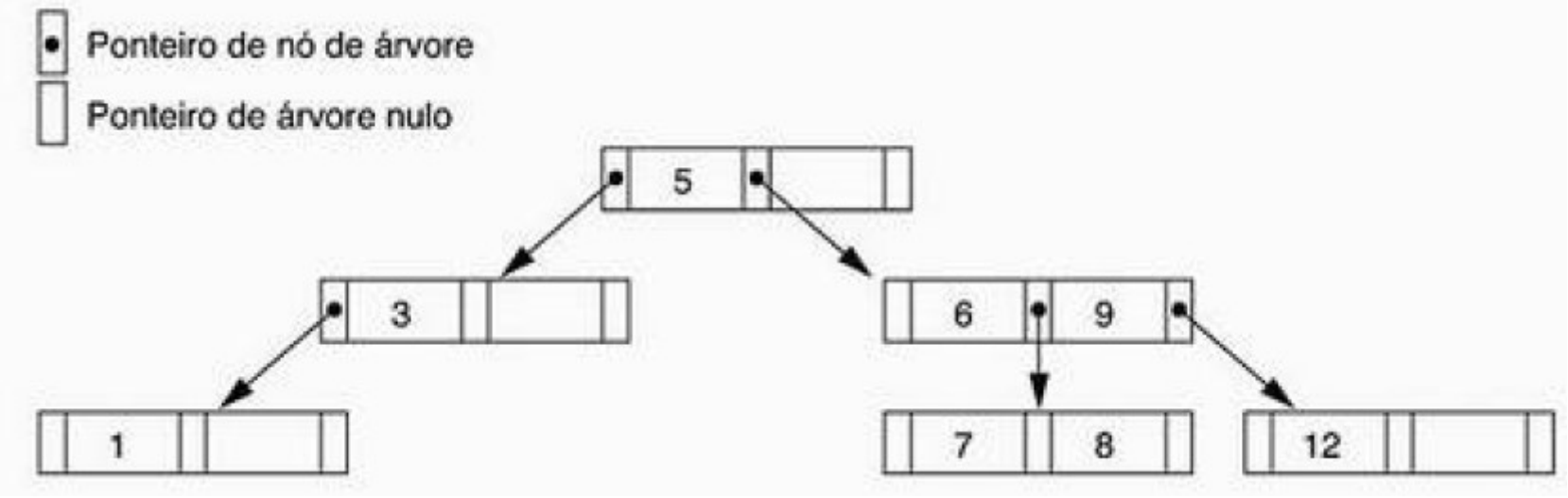
Árvore de pesquisa

- Um nó com apontamentos para subárvores abaixo



- Restrições
  - Em cada nó $K_1 < K_2 < ... < K_{q-1}$
  - Para todos os valores X na subárvore apontada por P_i, temos
    - $X < K_i$ para i=1
    - $K_{i-1} < X < K_i$ para $1 < i < q$
    - $K_{i-1} < X$ para i=q

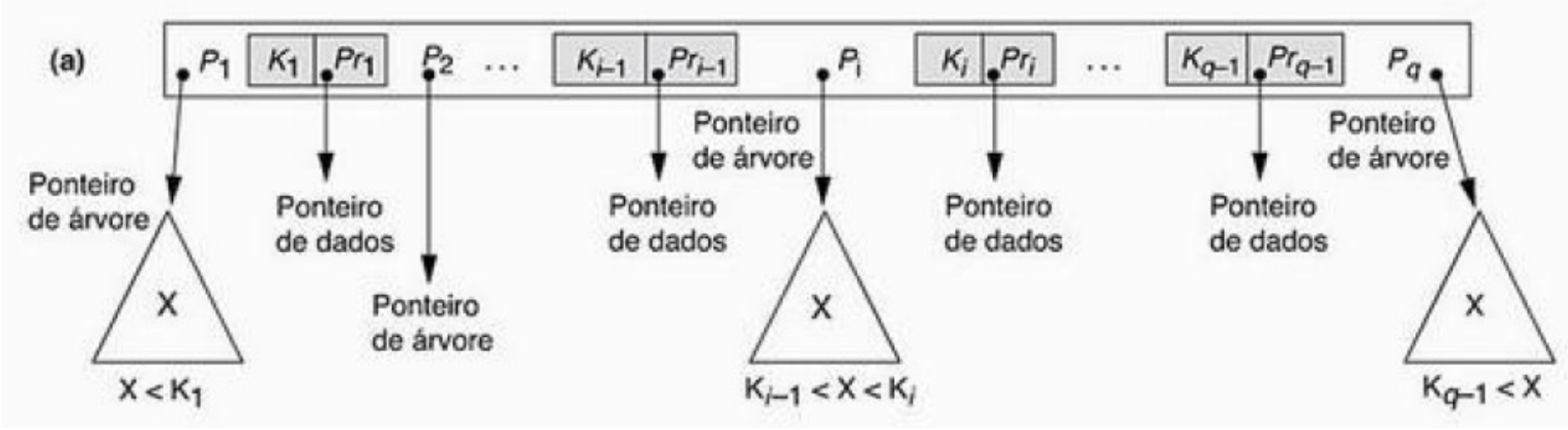
Exemplo árvore de pesquisa de ordem p=3



- Árvore de pesquisa como mecanismo de busca de registros armazenados em um arquivo em disco
  - Os valores na árvore podem ser os valores de um dos campos do arquivo, chamado campo de pesquisa (ou campo de índice)
  - Cada valor de chave da árvore é associado a um ponteiro para o registro no arquivo de dados
  - Um ponteiro aponta para um bloco de disco contendo o registro

Árvore B

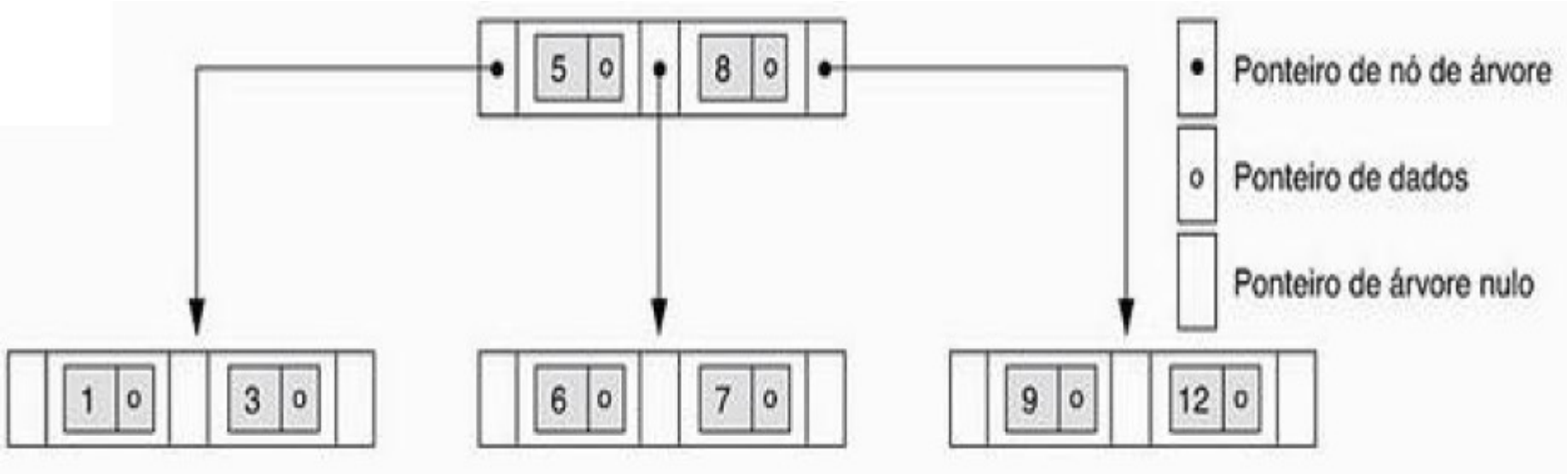
- Usada como índices de múltiplos níveis dinâmico
- Diferente da árvore de pesquisa, as restrições da árvore B garantem
  - balanceamento
  - reduz desperdício de espaço na exclusão
- Ponteiros para blocos de dados nos nós internos e folhas
- Um nó com q – 1 valores de pesquisa, q – 1 ponteiros de dados e q ponteiros de árvores



Árvore B

- Restrições
  - Cada nó interno $<P_1, <K_1, Pr_1>, P_2, <K_2, Pr_2>, ..., <K_{q-1}, Pr_{q-1}>, P_q>$ onde $q<=p$. P_i é um ponteiro para outro nó da árvore, Pr_i é um ponteiro de dados para um registro igual a K_i ou bloco do arquivo de dados que contém o registro
  - Em cada nó $K_1 < K_2 < ... < K_{q-1}$
  - Para todos os valores de campo da chave de pesquisa X na subárvore apontada por P_i, temos
    - $X < K_i$ para $i=1$
    - $K_{i-1} < X < K_i$ para $1 < i < q$
    - $K_{i-1} < X$ para $i=q$
  - Cada nó tem no máximo P ponteiros de árvores
  - Cada nó, exceto os nós raiz e folha, tem pelo menos teto(p/2) ponteiros de árvore. O nó raiz tem pelo menos 2 ponteiros de árvore, a menos que seja o único nó
  - Um nó com q ponteiros de árvore, $q<=p$, tem $q – 1$ valores de campo de chave de pesquisa (logo, $q – 1$ ponteiros de dados)
  - Todos os nós folhas estão no mesmo nível e têm a mesma estrutura dos nós internos

Exemplo árvore B de ordem p=3



- Ordem de inserção 8, 5, 1, 7, 3, 12, 9, 6

Árvore B

- Inserção no nó raiz cheio
  - Divide-se o nó raiz em dois nós no nível 1
  - Nó raiz fica com o valor do meio
  - Demais valores dividido por igual entre os outros dois nós
- Inserção num nó não raiz cheio
  - Divide-se esse nó em dois nós no mesmo nível
  - A entrada do meio é movida para o nó pai junto com os dois ponteiros para os novos nós divididos
  - Se o nó pai estiver cheio, ele também é dividido, propagando-se por todo caminho até o nó raiz, criando-se um novo nível se a raiz for dividida.
- Na exclusão, se um nó ficar com menos da metade cheio, ele é combinado com seus nós vizinhos, e propagando até raiz, reduzindo níveis na árvore B.
- Na estrutura de um nó pode ter outras informações, como quantidade de entradas q e um ponteiro para o nó pai

Árvore B – exemplo 4

- Suponha um campo de pesquisa em uma árvore B para um campo de chave não ordenado

p=23

cada nó com 69% cheio

Logo, p * 0,69 = 23 * 0,69 = 16 ponteiros aproximadamente

15 valores de campo de chave de pesquisa

fo = 16 em média

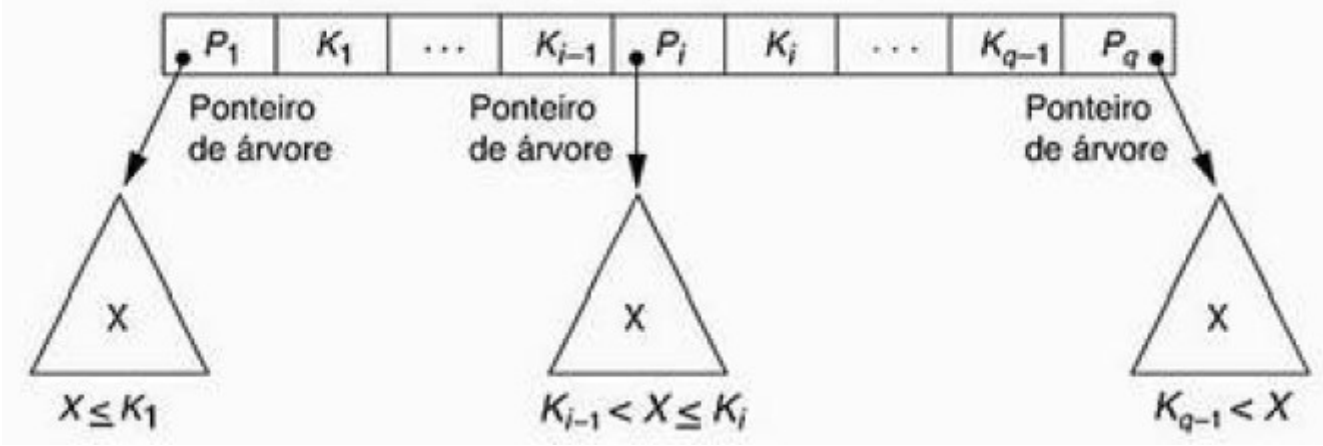
Calcular a quantidade de entradas para a árvore B de três níveis

Raiz	**1 nó**	**15 entradas de chave**	**16 ponteiros**
Nível 1	**16 nós**	**240 entradas de chave**	**256 ponteiros**
Nível 2	**256 nós**	**3.840 entradas de chave**	**4.096 ponteiros**
Nível 3	**4096 nós**	**61.440 entradas de chave**	

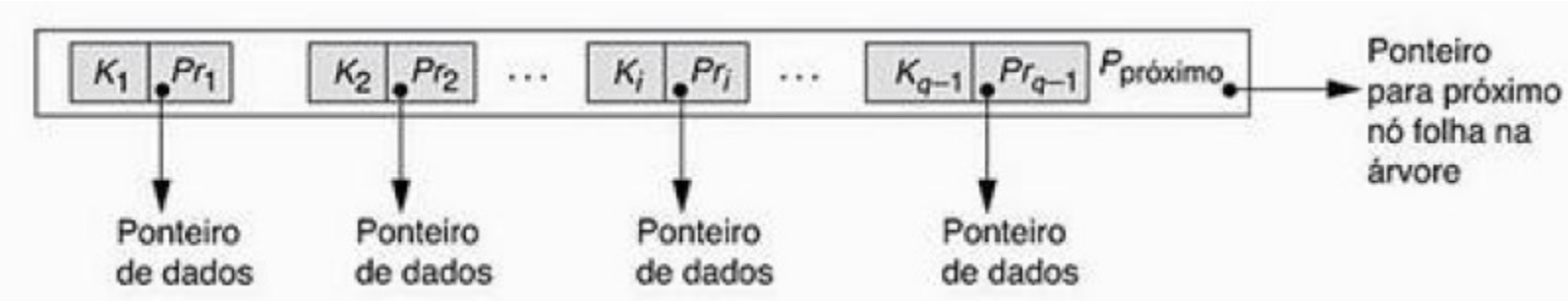
Para determinado tamanho de bloco, de ponteiro e de campo chave de pesquisa a árvore B de três níveis mantém **61.440+3.840+240+15=65.535** entradas (**blocos**) na média

Árvore B+

- Variação da árvore B para implementar índices de múltiplos níveis dinâmico
- Os ponteiros de dados são armazenados apenas nos nós folha da árvore
- Nó interno com q – 1 valores de pesquisa



- Nó folha com q – 1 valores de pesquisa e q – 1 ponteiros de dados



Árvore B^+

- **Para guiar uma pesquisa na árvore B+ alguns valores de pesquisa estão tanto em nós internos como nós folhas**

- **Restrições do nó interno**
  - Cada nó interno $<P_1, K_1, P_2, K_2, ..., P_{q-1}, K_{q-1}, P_q>$ onde $q<=p$ e cada P_i é um ponteiro de árvore
  - Em cada nó interno $K_1 < K_2 < ... < K_{q-1}$
  - Para todos os valores de campo de pesquisa X na subárvore apontada por P_i, temos
    - $X <= K_i$ para $i=1$
    - $K_{i-1} < X <= K_i$ para $1 < i < q$
    - $K_{i-1} < X$ para $i=q$
  - Cada nó interno tem no máximo P ponteiros de árvores
  - Cada nó interno, exceto a raiz, tem pelo menos teto(p/2) ponteiros de árvore. O nó raiz tem pelo menos 2 ponteiros de árvore, se for um nó interno
  - Um nó interno com q ponteiros, $q<=p$, tem $q – 1$ valores de campo de chave de pesquisa

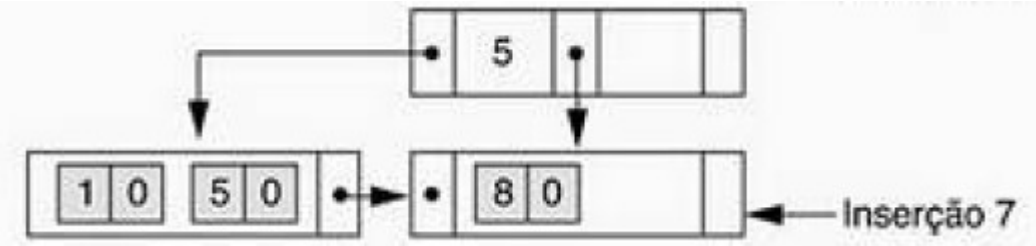
Árvore B^+

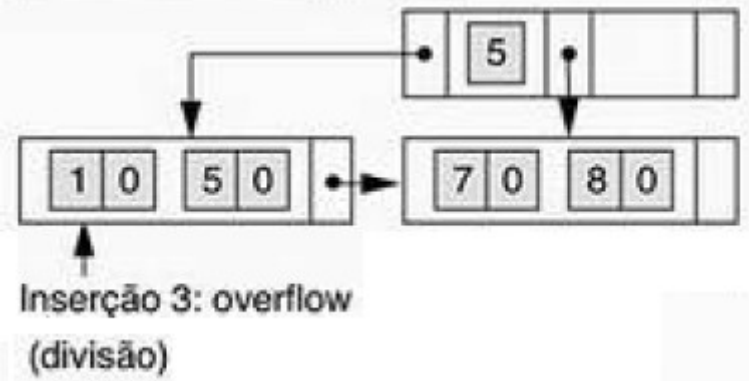
- **Restrições do nó folha**
  - Cada nó folha $<<K_1, Pr_1>, <K_2, Pr_2>, ..., <K_{q-1}, Pr_{q-1}>, P_{proximo}>$ onde $q<=p$ e cada Pr_i é um ponteiro de dados e $P_{proximo}$ aponta para o próximo nó folha
  - Em cada nó folha $K_1 <= K_2 ..., K_{q-1}$, $q<=p$
  - Cada Pr_i é um ponteiro de dados que aponta para o registro cujo valor do campo de pesquisa é K_i, ou bloco do arquivo de dados que contém o registro (ou bloco de ponteiros de registro que aponta para registros cujo valor do campo de pesquisa é K_i se o campo de pesquisa não for uma chave)
  - Cada nó folha tem pelo menos teto(p/2) valores
  - Todos os nós folhas estão no mesmo nível

- **O ponteiro$_{proximo}$ (para um nó folha mais a direita) permite acesso ordenado aos registros de dados no campo de índice**

Exemplo árvore B^+ com p=3 e p_{folha}=2

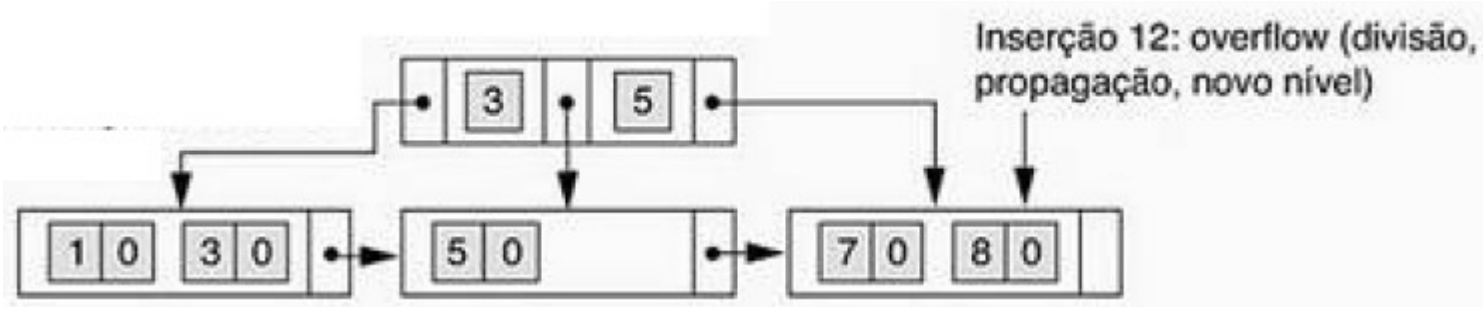
- **Ordem de inserção 8, 5, 1, 7, 3, 12, 9, 6**

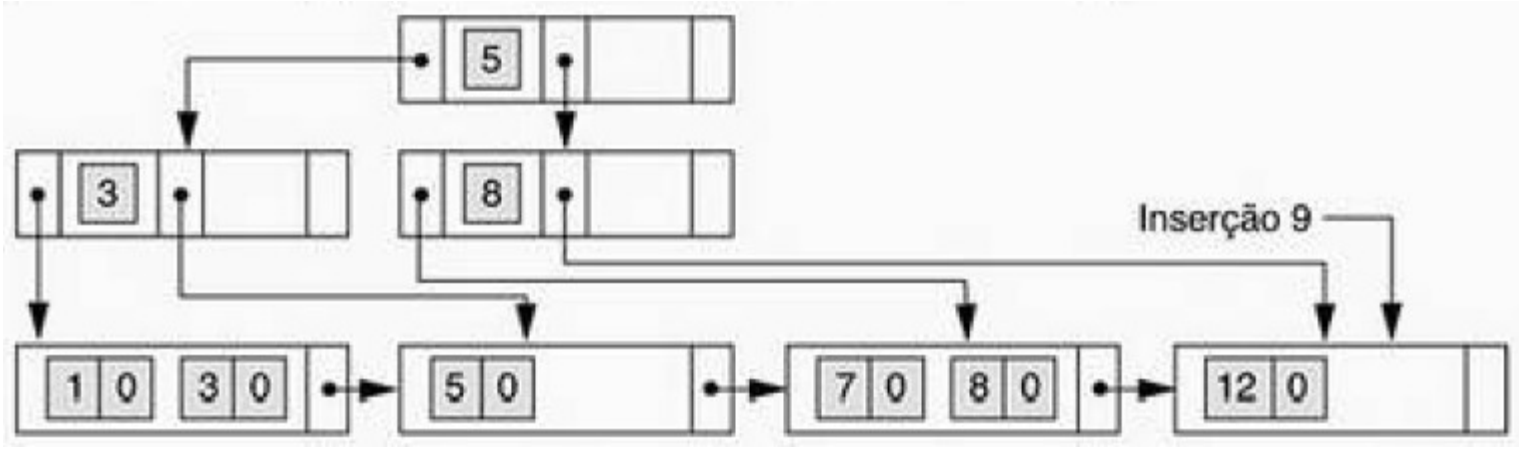




Exemplo árvore B^+ com p=3 e p_{folha}=2

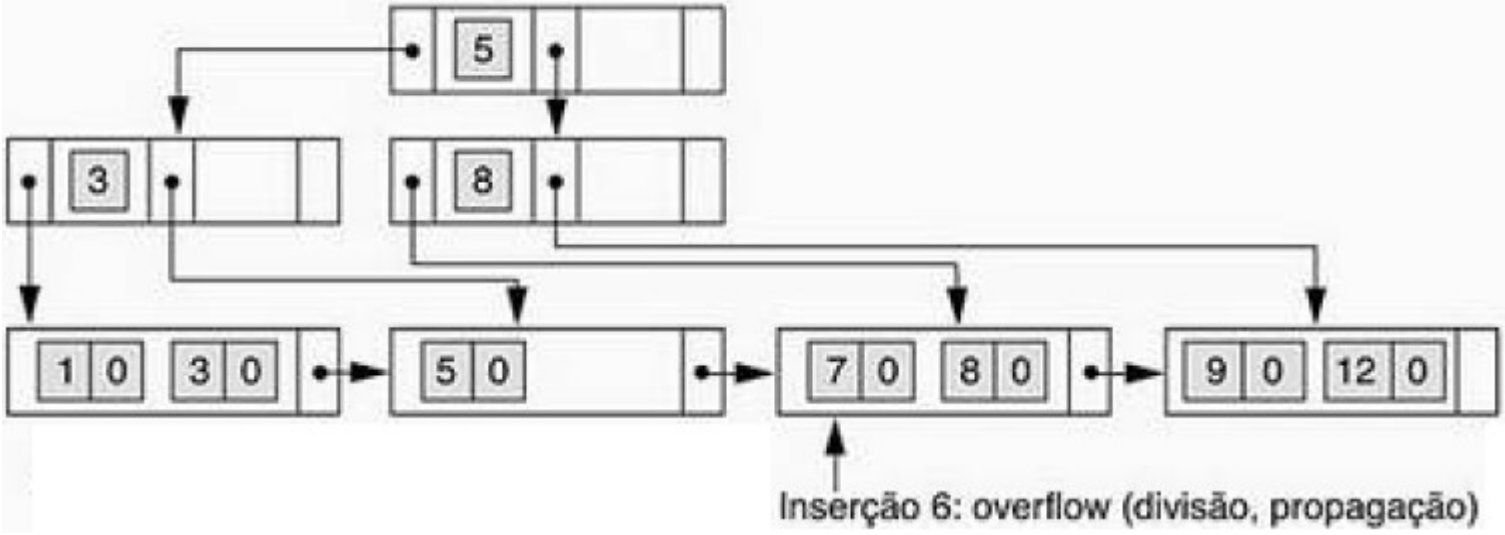
- **Ordem de inserção 8, 5, 1, 7, 3, 12, 9, 6**

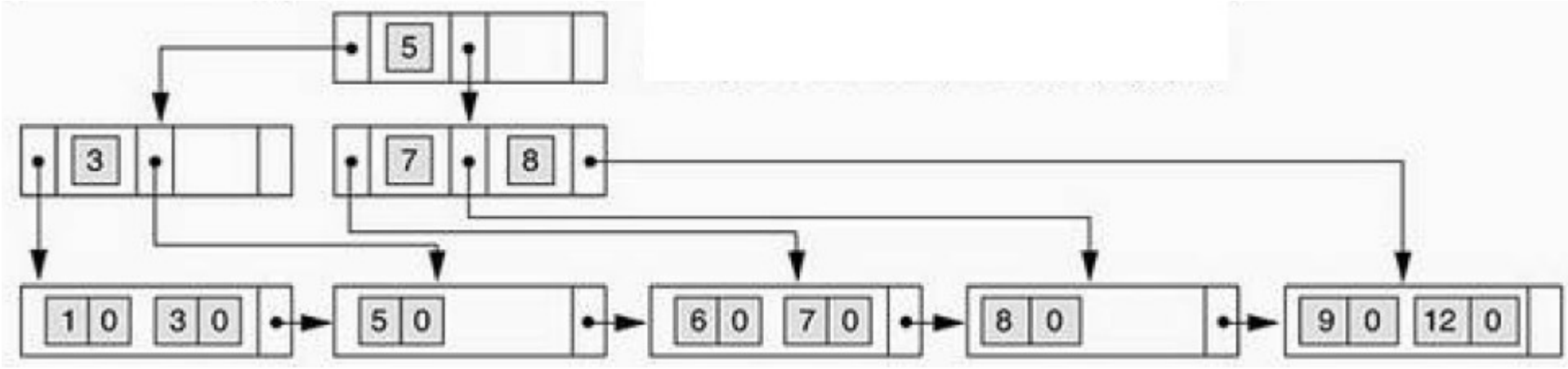




Exemplo árvore B^+ com p=3 e p_{folha}=2

- **Ordem de inserção 8, 5, 1, 7, 3, 12, 9, 6**





Árvore B^+ – exemplo 5

- Suponha um campo de chave de pesquisa seja V=9 bytes de extensão, o tamanho do bloco seja B=512 bytes, um ponteiro de registro seja Pr=7 bytes e um ponteiro de bloco seja P=6 bytes. Um nó interno da B^+ pode ter até p ponteiros de árvore e p – 1 valores de campo de pesquisa; estes precisam caber num único bloco.

Calcular a ordem p de um nó interno da B^+

$(p * P) + ((p - 1) * V) <= B$

$(p * 6) + ((p - 1) * 9) <= 512$

$(15*p) <= 521$

Escolhendo um valor de p que seja o maior que satisfaça a desigualdade, logo p=34

Calcular a ordem p de um nó folha da B^+

$(p_{folha} * (Pr + V)) + P <= B$

$(p_{folha} * (7 + 9)) + 6 <= 512$

$(p_{folha} * (16)) <= 506$

Escolhendo um valor de p_{folha} que seja o maior que satisfaça a desigualdade, logo p=31

Árvore B^+ – exemplo 6

➔ Calcular a quantidade aproximada de entradas para uma B^+ de 3 níveis considerando que cada nó está 69% cheio.

Na média cada nó interno terá 34 * 0,69 ou, aproximadamente 23 ponteiros, logo 22 valores.

Na média cada nó folha terá 31 * 0,69 ou, aproximadamente 21 ponteiros de registros de dados,

Em cada nível a média de entradas será

Raiz	**1 nó**	**22 entradas de chave**	**23 ponteiros**
Nível 1	**23 nós**	**506 entradas de chave**	**529 ponteiros**
Nível 2	**529 nós**	**11.638 entradas de chave**	**12.167 ponteiros**
$Nível_{folha}$	**12.167 nós**	**255.507 ponteiros de registros de dados**	

Para tamanho de bloco, tamanho de ponteiro e tamanho do campo de pesquisa a árvore B^+ de três níveis mantém até **255.507** (12.167*21) ponteiros de registros de dados, contra **65.535** em uma B. Isto justifica o uso de B^+ como índices para bancos de dados

Outros assuntos sobre índices

- Índices sobre chaves múltiplas

- Outros tipos de índices